

PBA: 2.

Data Visualization

Fall 2026

Jaewon (“Jay-one”) Yoo

National Tsing Hua University

Roadmap

1. **A first look at data, and a first plot**
 - Gapminder: one row is a country in a year
2. **[First glance at] visualization for exploratory analysis**
 - Building plots by layers
 - Geoms (i.e., geometric objects) are functions!
 - Adding informative labels
 - Mapping versus setting aesthetics
3. **Histograms/boxplots and grouped data**
 - Histogram and boxplots
 - Grouped data

Rest of the class: In-class exercise E1 (Gapminder)

Reference slides on R basics (types, vectors, data frames, operators, functions) are posted separately. Read them alongside AAG Ch. 4–6.

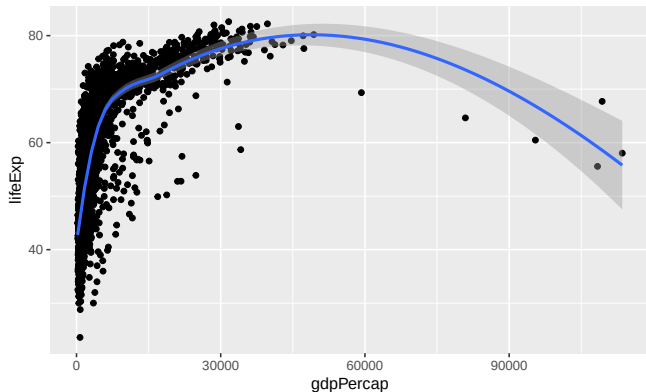
Gapminder Data

```
1 > library(ggplot2)
2 > gapminder <- gapminder::gapminder
3 > gapminder
4 # A tibble: 1,704 x 6
5   country      continent  year lifeExp      pop gdpPercap
6   <fct>        <fct>    <int>  <dbl>   <int>   <dbl>
7 1 Afghanistan Asia      1952   28.8  8425333    779.
8 2 Afghanistan Asia      1957   30.3  9240934    821.
9 3 Afghanistan Asia      1962   32.0 10267083    853.
10 # ... 1,701 more rows
```

- We now use a table supplied by a package, instead of creating one by hand.
- One row is a **country in a particular year**.
- 142 countries, 12 years each (1952–2007). We use the full table throughout today's plots.
- <fct>: a factor, used for categories; <int>: integer; <dbl>: numeric.

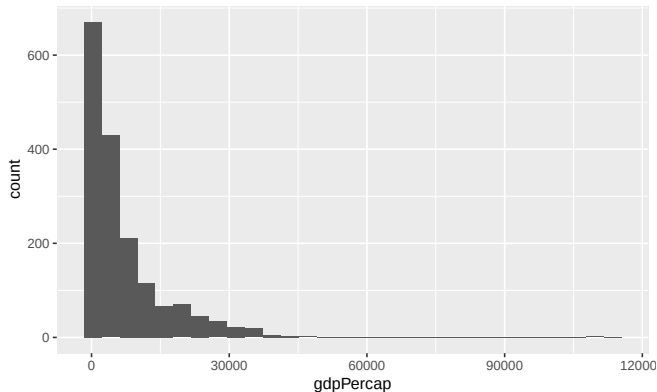
Life Expectancy and GDP per Capita

```
1 > ggplot(gapminder, mapping = aes(x = gdpPerCap, y = lifeExp)) +  
2 +   geom_point() + geom_smooth(method = "loess")
```



A Histogram of GDP per Capita

```
1 > ggplot(gapminder, mapping = aes(x = gdpPercap)) +  
2 +   geom_histogram()
```



Two lines of code drew that.

What did each piece do?

The rest of today: the parts of a ggplot you will use every week, then time to build your own.

1/ Visualization for Exploratory Analysis

Midwest Data

```
1 > ggplot2::midwest ## If needed: install.packages("ggplot2")
2
3 # A tibble: 437 × 28
4   PID county state area poptotal popdensity popwhite popblack popamerindian popasian popother
5   <int> <chr> <chr> <dbl> <int> <dbl> <int> <int> <int> <int> <int>
6 1 561 ADAMS IL 0.052 66090 1271. 63917 1702 98 249 124
7 2 562 ALEXAN IL 0.014 10626 759 7054 3496 19 48 9
8 3 563 BOND IL 0.022 14991 681. 14477 429 35 16 34
9 4 564 BOONE IL 0.017 30806 1812. 29344 127 46 150 1139
10 5 565 BROWN IL 0.018 5836 324. 5264 547 14 5 6
11 6 566 BUREAU IL 0.05 35688 714. 35157 50 65 195 221
12 7 567 CALHOUN IL 0.017 5322 313. 5298 1 8 15 0
13 8 568 CARROLL IL 0.027 16805 622. 16519 111 30 61 84
14 9 569 CASS IL 0.024 13437 560. 13384 16 8 23 6
15 10 570 CHAMPA IL 0.058 173025 2983. 146506 16559 331 8033 1596
16 # i 427 more rows
17 # i 17 more variables: percwhite <dbl>, percblack <dbl>, percamerindian <dbl>, percasian <dbl>,
18 # percother <dbl>, popadults <int>, perchsds <dbl>, percollege <dbl>, percprof <dbl>,
19 # poppovertyknown <int>, percpovertyknown <dbl>, percbelowpoverty <dbl>,
20 # percchildbelowpovert <dbl>, percadultpoverty <dbl>, percelderlypoverty <dbl>, inmetro <int>,
21 # category <chr>
22 # i Use `print(n = ...)` to see more rows
```

Building Up a Graph in Pieces

- Create a ggplot object and direct it to the correct data:

```
1 > p <- ggplot(data = midwest)
```

- **Mapping:** tell ggplot what visual aesthetics correspond to which variables:

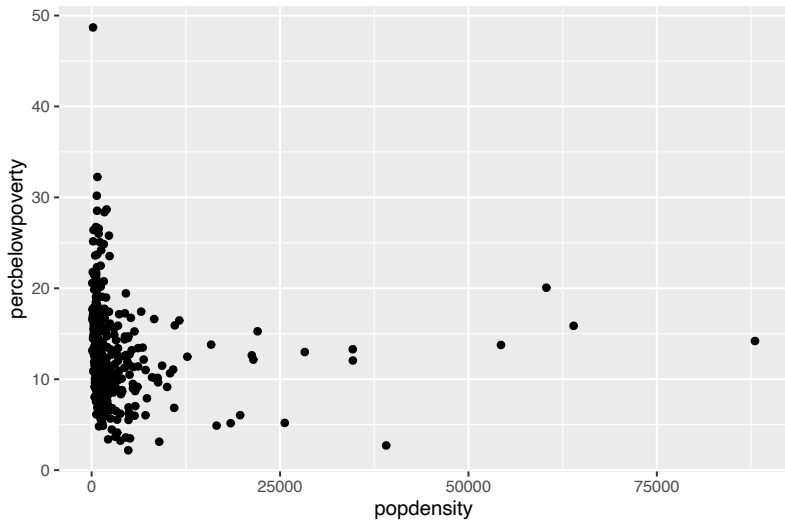
```
1 > p <- ggplot(data = midwest,  
2 +           mapping = aes(x = popdensity,  
3 +                         y = percbelowpoverty))
```

- Other aesthetic mapping parameters: color, shape, size, etc.

Adding a geom Layer

```
1 > ggplot(data = midwest,  
2 +       mapping = aes(x = popdensity,  
3 +                     y = percbelowpoverty)) +  
4 +       geom_point()
```

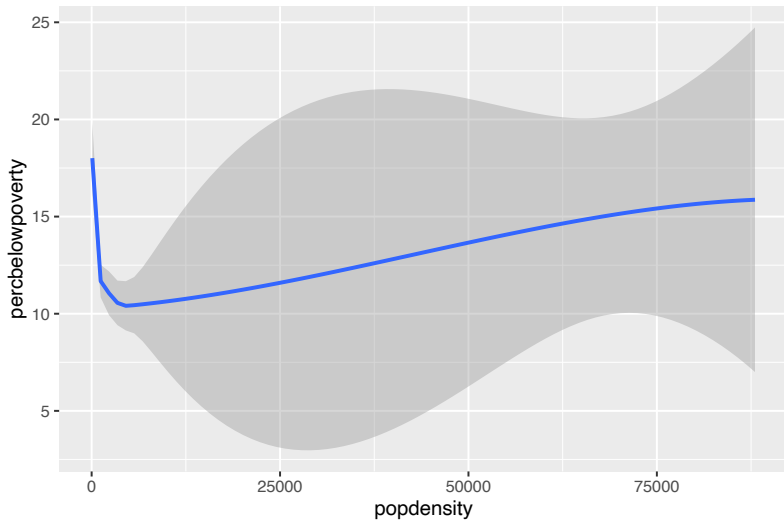
Adding a geom Layer



Trying a New geom

```
1 > ggplot(data = midwest,  
2 +       mapping = aes(x = popdensity,  
3 +                     y = percbelowpoverty)) +  
4 +       geom_smooth()
```

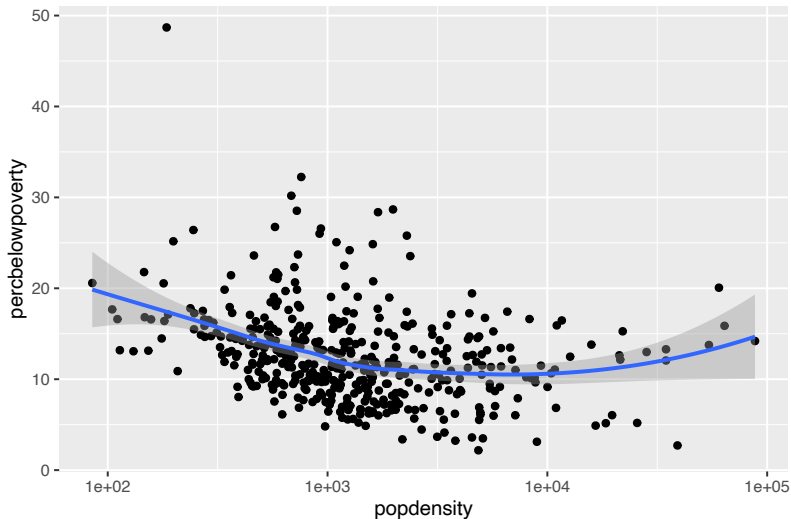
Trying a New geom



Layering geom is Additive!

```
1 > ggplot(data = midwest,  
2 +       mapping = aes(x = popdensity,  
3 +                     y = percbelowpoverty)) +  
4 +     geom_point() +  
5 +     geom_smooth() +  
6 +     scale_x_log10() # Log scale for the skewed x variable
```

Layering geom is Additive!

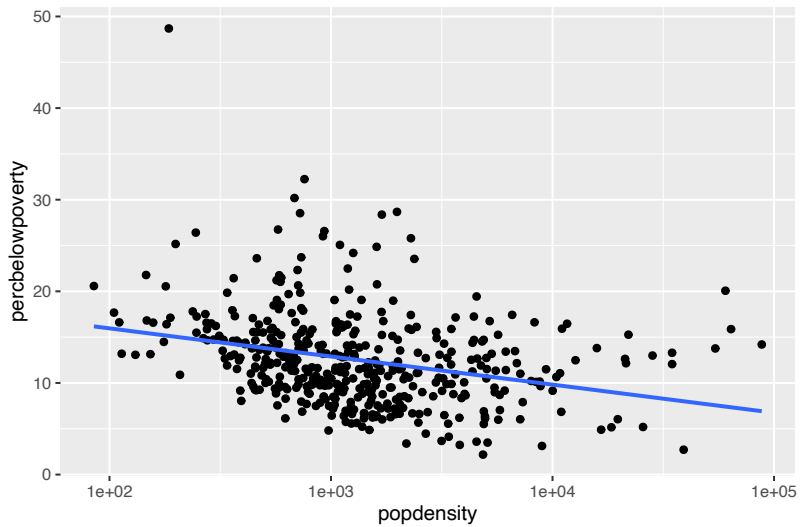


Geoms are Functions

- Geoms can take input arguments:

```
1 > ggplot(data = midwest,  
2 +       mapping = aes(x = popdensity,  
3 +                     y = percbelowpoverty)) +  
4 +       geom_point() +  
5 +       geom_smooth(method = "lm", se = F) +  
6 +       scale_x_log10()
```

↪ Tells `geom_smooth()` to do a linear fit with no error region.

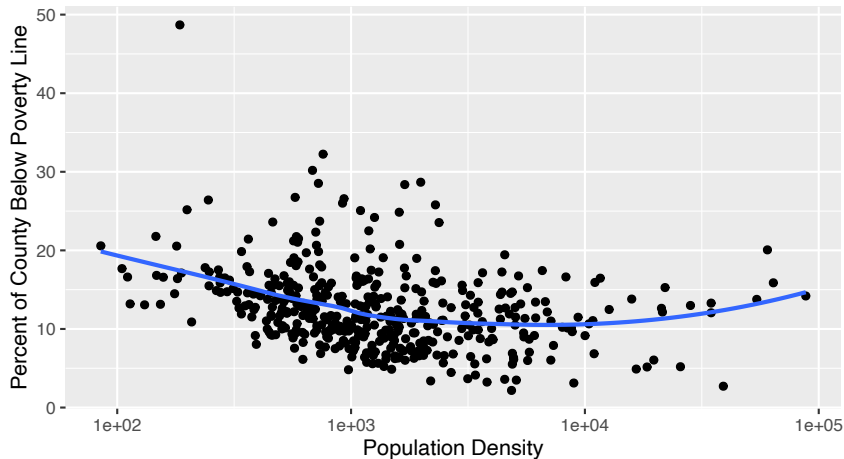


Adding Informative Labels

```
1 > ggplot(data = midwest,  
2 +       mapping = aes(x = popdensity,  
3 +                     y = percbelowpoverty)) +  
4 +     geom_point() +  
5 +     geom_smooth(method = "loess", se = F) +  
6 +     scale_x_log10() +  
7 +     labs(x = "Population Density",  
8 +          y = "Percent of County Below Poverty Line",  
9 +          title = "Poverty and Population Density",  
10 +          subtitle = "Among Counties in the Midwest",  
11 +          caption = "Source: US Census, 2000")
```

Poverty and Population Density

Among Counties in the Midwest

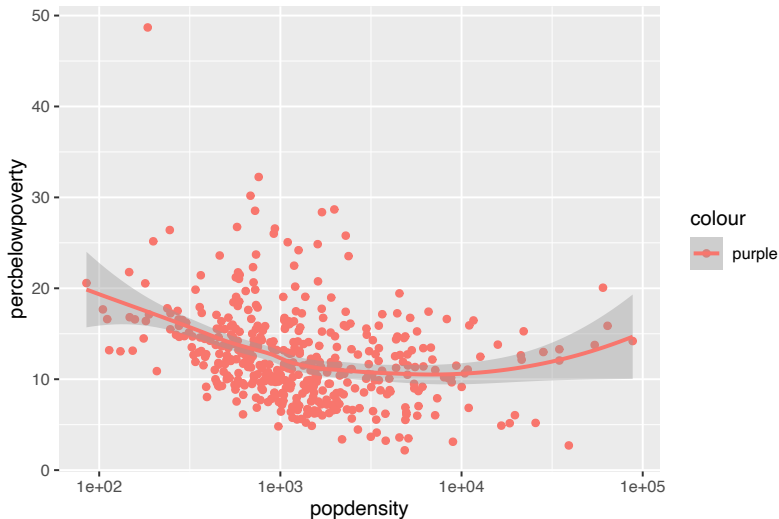


Source: US Census, 2000

Mapping vs. Setting Aesthetics

```
1 > ggplot(data = midwest,  
2 +       mapping = aes(x = popdensity,  
3 +                     y = percbelowpoverty,  
4 +                     color = "purple")) +  
5 +   geom_point() +  
6 +   geom_smooth() +  
7 +   scale_x_log10()
```

Wait, What Happened?



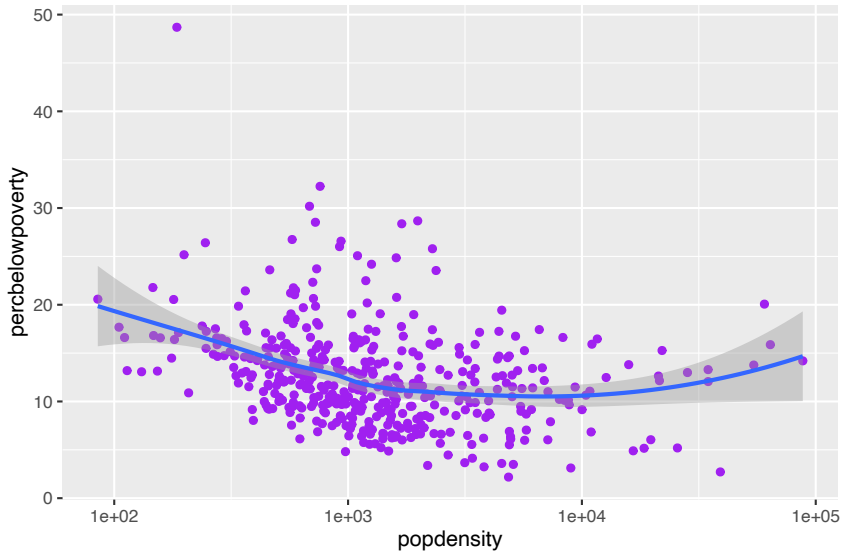
Wait, What Happened?

- Mapping always refers to variables!
 ↪ If passed a value other than a variable name, ggplot will implicitly create a variable with that value (in this case “purple” that is constant.)

Setting Aesthetics

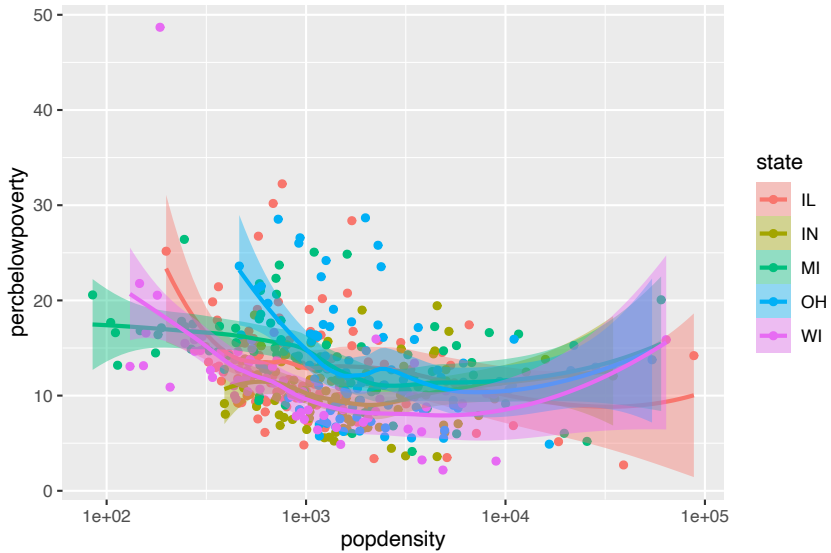
- Set the color outside the mapping (i.e., `aes()`) format:

```
1 > ggplot(data = midwest,  
2 +       mapping = aes(x = popdensity,  
3 +                     y = percbelowpoverty)) +  
4 +     geom_point(color = "purple") +  
5 +     geom_smooth() +  
6 +     scale_x_log10()
```



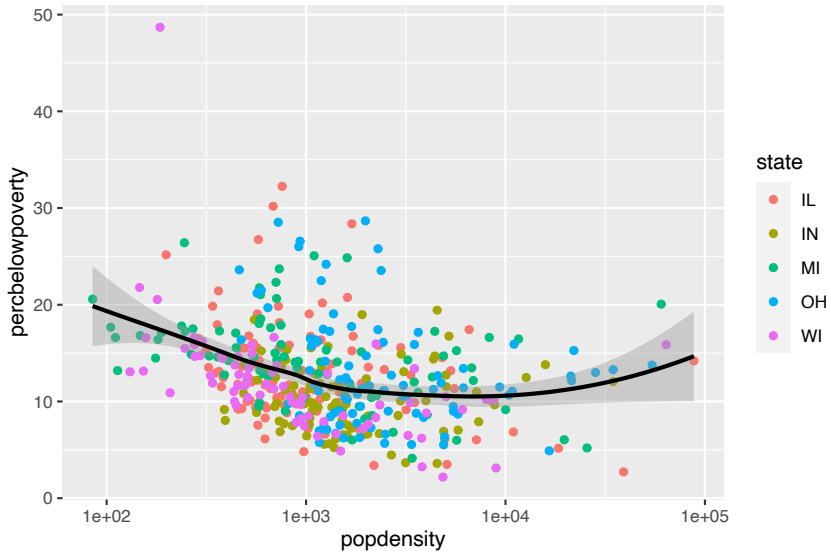
Mapping More Aesthetics

```
1 > ggplot(data = midwest,  
2 +       mapping = aes(x = popdensity,  
3 +                     y = percbelowpoverty,  
4 +                     color = state,  
5 +                     fill = state)) +  
6 +   geom_point() +  
7 +   geom_smooth() +  
8 +   scale_x_log10()
```



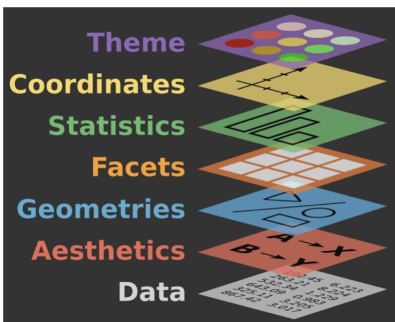
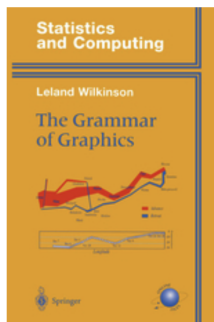
Mapping Can be Done on a Per Geom Basis

```
1 > ggplot(data = midwest,  
2 +       mapping = aes(x = popdensity,  
3 +                     y = percbelowpoverty)) +  
4 +   geom_point(mapping = aes(color = state)) +  
5 +   geom_smooth(color = "black") +  
6 +   scale_x_log10()
```



ggplot2: The Grammar of Graphics

“...the grammar tells us that a statistical graphic is a mapping from data to aesthetic attributes (colour, shape, size) of geometric objects (points, lines, bars). The plot may also contain statistical transformations of the data and is drawn on a specific coordinate system” – Leland Wilkinson



2/ Histograms, Boxplots, and Grouped Data

Midwest Data Redux

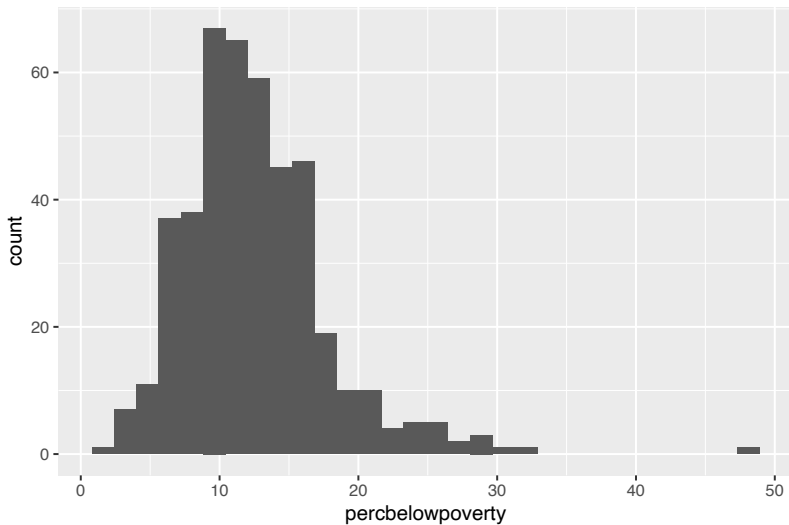
```
1 > ggplot2::midwest ## If needed: install.packages("ggplot2")
2
3 # A tibble: 437 × 28
4   PID county state area poptotal popdensity popwhite popblack popamerindian popasian popother
5   <int> <chr> <chr> <dbl> <int> <dbl> <int> <int> <int> <int> <int>
6 1 561 ADAMS IL 0.052 66090 1271. 63917 1702 98 249 124
7 2 562 ALEXAN IL 0.014 10626 759 7054 3496 19 48 9
8 3 563 BOND IL 0.022 14991 681. 14477 429 35 16 34
9 4 564 BOONE IL 0.017 30806 1812. 29344 127 46 150 1139
10 5 565 BROWN IL 0.018 5836 324. 5264 547 14 5 6
11 6 566 BUREAU IL 0.05 35688 714. 35157 50 65 195 221
12 7 567 CALHOUN IL 0.017 5322 313. 5298 1 8 15 0
13 8 568 CARROLL IL 0.027 16805 622. 16519 111 30 61 84
14 9 569 CASS IL 0.024 13437 560. 13384 16 8 23 6
15 10 570 CHAMPA IL 0.058 173025 2983. 146506 16559 331 8033 1596
16 # i 427 more rows
17 # i 17 more variables: percwhite <dbl>, percblack <dbl>, percamerindian <dbl>, percasian <dbl>,
18 # percother <dbl>, popadults <int>, perchsds <dbl>, percollege <dbl>, percprof <dbl>,
19 # poppovertyknown <int>, percpovertyknown <dbl>, percbelowpoverty <dbl>,
20 # percchildbelowpovert <dbl>, percadultpoverty <dbl>, percelderlypoverty <dbl>, inmetro <int>,
21 # category <chr>
22 # i Use `print(n = ...)` to see more rows
```

Histograms

- **Histograms** show where there are more or fewer observations of a numeric variable.

```
1 > require(ggplot2)
2 > ggplot(data = midwest,
3 +       mapping = aes(x = percbelowpoverty)) +
4 +       geom_histogram()
```

- Split up range of variable into bins, count how many are in each bin.
- y aesthetic calculated automatically.



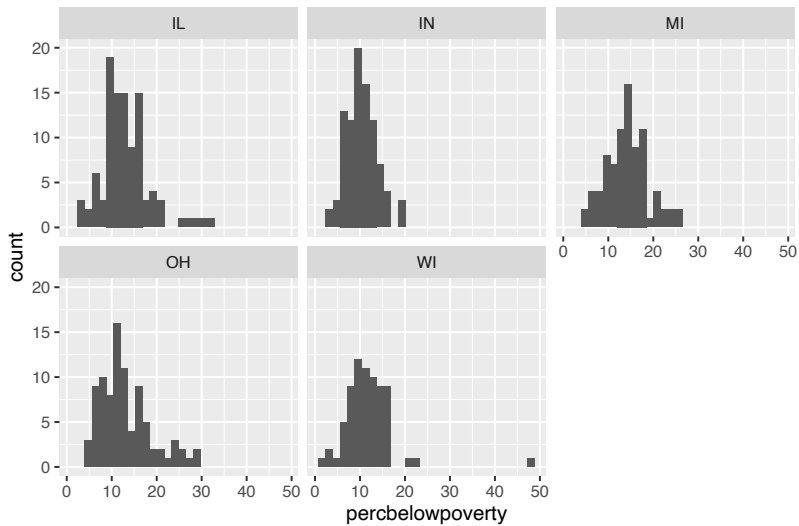
Take away: The distribution peaks near 10% below the poverty line, with a long right tail across counties.

Creating Small Multiples with Facets

- Small multiples: a series of similar graphs with the same scale/axes to help with comparing different partitions of a dataset

```
1 > ggplot(data = midwest,  
2 +       mapping = aes(x = percbelowpoverty)) +  
3 +       geom_histogram() +  
4 +       facet_wrap(~ state)
```

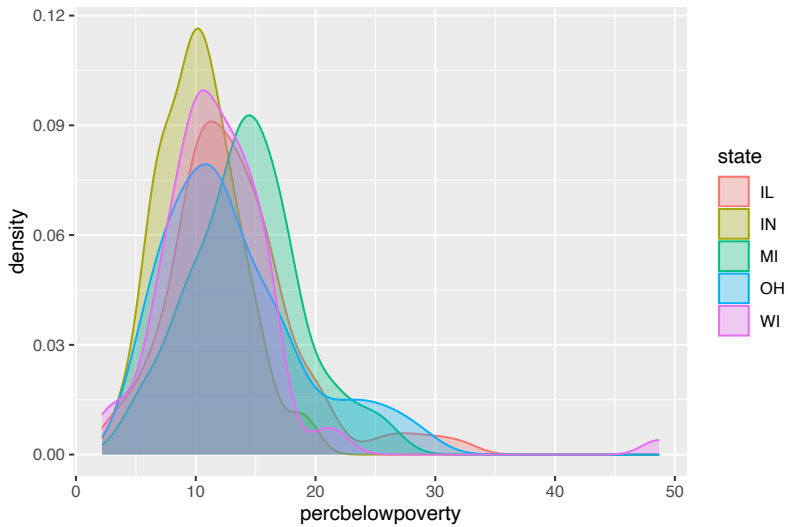
- The variable used for faceting/splitting must be placed on the right-hand side of the tilde (~) operator
 - We'll see more of the ~ variable syntax (called a formula) throughout the semester!



Density as Alternative to Histogram

- A kernel density plot is a smoothed version of a histogram and slightly easier to overlay.

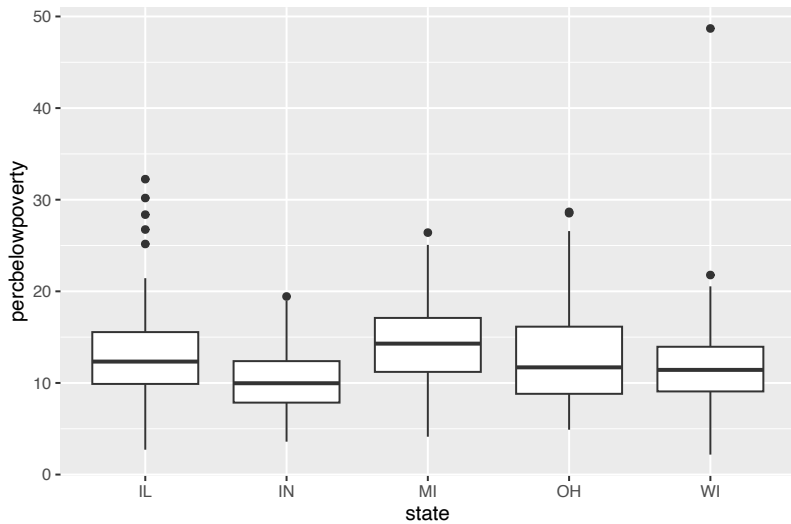
```
1 > ggplot(data = midwest,  
2 +       mapping = aes(x = percbelowpoverty,  
3 +                     fill = state, color = state)) +  
4 +       geom_density(alpha = 0.3)
```



Boxplots

- Boxplots are another way to compare distributions across discrete groups.

```
1 > ggplot(data = midwest,  
2 +       mapping = aes(x = state,  
3 +                     y = percbelowpoverty)) +  
4 +       geom_boxplot()
```



More on Boxplots

- “Box” spans the middle 50%: from Q_1 (25th percentile) to Q_3 (75th percentile).
 - 25% of the data above the box, 25% below
 - Height of the box is the interquartile range: $IQR = Q_3 - Q_1$
- Horizontal line in the box is the median
 - 50% of the data above the median, 50% below
- “Whiskers” extend to observed values:
 - Lowest value at least $Q_1 - 1.5 IQR$; highest value at most $Q_3 + 1.5 IQR$
 - Points beyond the whiskers are plotted separately as potential outliers

Visualizing grouped data accurately is key to conveying the right insights and avoiding misleading interpretation.

Gapminder Data

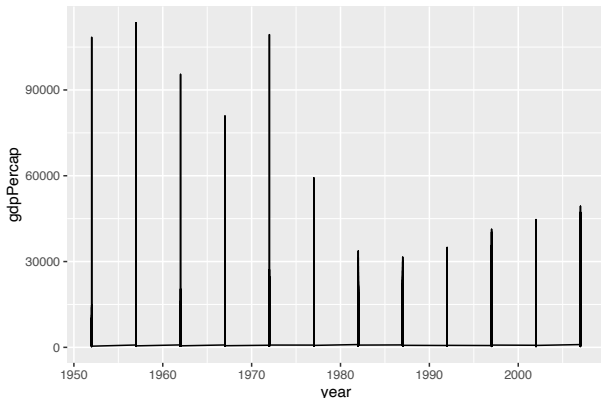
```
1 > require(gapminder)
2 > dplyr::glimpse(gapminder)
3
4 Rows: 1,704
5 Columns: 6
6 $ country   <fct> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", "Afgha...
7 $ continent <fct> Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, ...
8 $ year      <int> 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, 2002, ...
9 $ lifeExp   <dbl> 28.801, 30.332, 31.997, 34.020, 36.088, 38.438, 39.854, 40.822, 41...
10 $ pop       <int> 8425333, 9240934, 10267083, 11537966, 13079460, 14880372, 12881816...
11 $ gdpPercap <dbl> 779.4453, 820.8530, 853.1007, 836.1971, 739.9811, 786.1134, 978.01...
```

- Let's say we are interested in learning about the trend in income in different countries.
 - ↪ Use `geom_line()` to create a line plot.

Let's plot the trend in income.. wait what?

```
1 > ggplot(data = gapminder,  
2 +       mapping = aes(x = year,  
3 +                       y = gdpPercap)) +  
4 +       geom_line() # Didn't specify how to group the lines..
```

Let's plot the trend in income.. wait what?

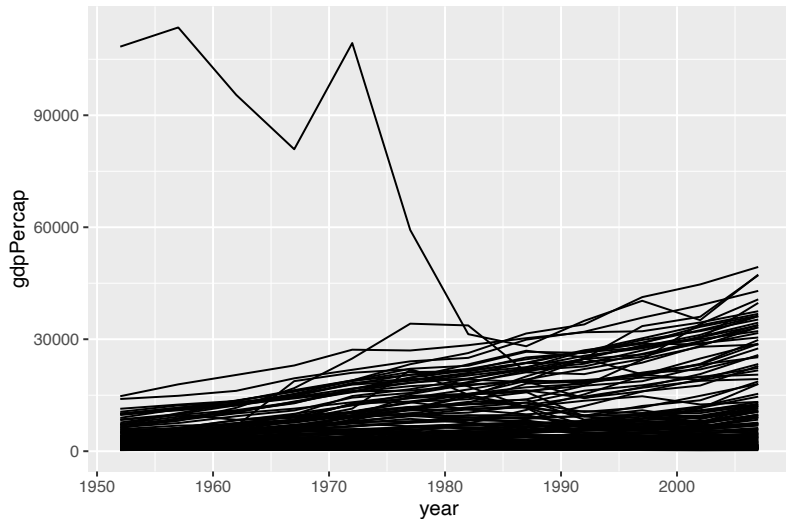


- `geom_line()` connected points from different countries in the same year.
 - What we wanted was to connect the points from same countries in different years (i.e., time trends)!

Tell `geom_line()` how to group the lines

```
1 > ggplot(data = gapminder,  
2 +       mapping = aes(x = year,  
3 +                       y = gdpPercap)) +  
4 +       geom_line(mapping = aes(group = country))
```

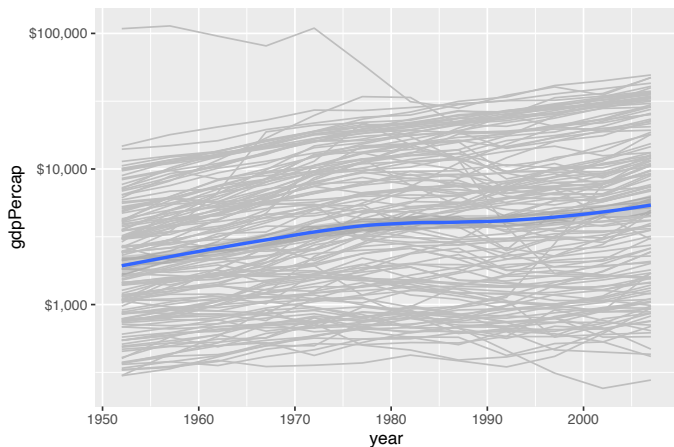
Tell `geom_line()` how to group the lines



Adjusting scales and adding a trendline

```
1 > ggplot(data = gapminder,  
2 +       mapping = aes(x = year,  
3 +                     y = gdpPercap)) +  
4 +   geom_line(mapping = aes(group = country), color = "gray") +  
5 +   geom_smooth(method = "loess") +  
6 +   scale_y_log10(labels = scales::dollar)
```

Adjusting scales and adding a trendline



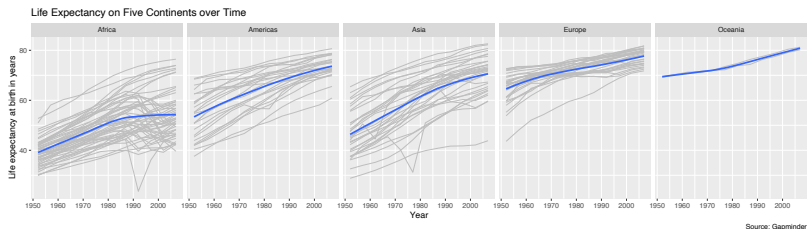
Q: What do we observe? Why do you think this is the case? (innovation? capital accumulation?)

In-Class Exercises: Background

- We want to find out the answer to the following question:
 - Q: how does life expectancy on five continents (e.g., Asia, Europe, Americas) change over time?
- The gapminder dataset contains some information about life expectancy across different continent, thus we could visualize the data to answer the question.
 - Some relevant variables:
 - `country`: name of the country
 - `continent`: name of the country's continent
 - `year`: year of the measurement, ranging from 1952 to 2007 in 5-year increments
 - `lifeExp`: life expectancy at birth, in years

In-Class Exercises: Task

- Recreate the following plot:



- The plot shows each country's life expectancy trajectory over time, broken out by continent with smoothed average lines overlayed for each continent.
 - To get started, we'll give you a few clues about what was done to create the plot in the following slide.

In-Class Exercises: Instructions

- The lines for each country use the color "gray"
- The width of the smoothed line is 1.1 (`linewidth = 1.1`) and the method used is the loess smoother.
- Turn off the standard errors.
- Make sure that the facets are all on one row. Look at the `facet_wrap` documentation if need help with this.
- Make sure that the labels are correctly specified.

Q: What does the visualization reveal? Explain what you observe verbally.

What Comes Next?

- Today: a table became a graph, layer by layer; the same data compared across distributions and groups.
- Next week: **data wrangling** with `dplyr`. Choose rows and columns, create variables, summarize groups, and draw barplots from the summaries.
- Before then: read the R basics reference slides and AAG Ch. 4–6, and finish swirl lessons 1–3. Assignment 1 is out today.